Лабораторна робота №12

Тема: Підсумковий mini-project: modern React, API, server/client state та архітектура застосунку.

Мета: самостійно застосувати знання, отримані під час курсу React, та побудувати невеликий завершений SPA-застосунок. Продемонструвати вміння розділяти server state та client/UI state, організовувати маршрути, API, repository, форми та повторно використовувані компоненти.

Ця лабораторна є підсумковою. На відміну від робіт №8–11, предметну область студент обирає самостійно із запропонованих варіантів.

Вимоги до звіту

1. Використовувати React 19.x та Node.js 22.22 або новішу версію.
2. Використовувати TypeScript.
3. Застосунок повинен бути SPA.
4. Використати React Router 8.
5. Працювати з REST API або іншим зовнішнім джерелом server state.
6. Використати TanStack Query v5 для server state.
7. Використати Zustand v5 для client/UI state.
8. Використати React Hook Form та Zod щонайменше для однієї форми.
9. Організувати код у зрозумілій структурі за одним із підходів: **feature-first** або **type-first**.
10. Реалізувати **Repository** для роботи з основною server-side сутністю.
11. Створити щонайменше один повторно використовуваний UI-компонент і використати його щонайменше у двох місцях.
12. Передбачити loading, error, empty та success states для асинхронних сценаріїв.
13. Реалізувати щонайменше одну значущу CRUD-операцію.
14. Не використовувати `alert`.
15. Для стилізації використати один із підходів: SCSS із дотриманням BEM, Bootstrap або Tailwind CSS.
16. Якщо обрано SCSS, дотримуватися BEM-патерну для класів.
17. Дотримуватися ESLint та TypeScript.
18. Код повинен бути розміщений в особистому GitHub репозиторії студента.
19. Дотримуватися Conventional Commits.

Версії та встановлення основних бібліотек

Для mini-project використовувати актуальний стек курсу:

- React 19.x;
- Node.js 22.22 або новішу версію;
- React Router 8;
- TanStack Query v5;
- Zustand v5;
- React Hook Form + Zod.

Для React Router 8 встановити пакет `react-router`:

```
npm install react-router
```

Для Declarative Mode імпортувати маршрути з `react-router`, а не з `react-router-dom`.

Основні бібліотеки можна встановити однією командою:

```
npm install react-router @tanstack/react-query zustand react-hook-form zod
@hookform/resolvers
```

Розподіл state

TanStack Query використовується для даних, джерелом яких є сервер, а Zustand — для стану, який належить самому клієнтському застосунку, наприклад кошика, обраного, watchlist, плану подорожі або UI-налаштувань.

Не потрібно дублювати один і той самий server state в обох інструментах.

Архітектурний підхід

У цій лабораторній **не потрібно реалізовувати повний FSD** і не потрібно дотримуватися конкретних назв папок.

Студент може обрати один із двох підходів:

1. **feature-first** — код групується переважно за функціональністю;
2. **type-first** — код групується переважно за технічною відповідальністю.

Головне — щоб відповідальність модулів була зрозумілою, а API-доступ не був змішаний з UI-кодом.

Приклад спрощеної feature-first структури:

```
src/
 ├──── app/
 ├──── pages/
 ├──── features/
 └──── shared/
```

Приклад спрощеної type-first структури:

```
src/
 ├──── app/
 ├──── pages/
 ├──── components/
 ├──── hooks/
 ├──── repositories/
 ├──── services/
 ├──── store/
 └──── types/
```

Це лише орієнтири. Структура може відрізнятися, якщо вона залишається зрозумілою.

Не потрібно створювати окрему папку або шар для кожного типу файлів лише заради шаблону.

Repository та API boundary

Repository є обов'язковим.

API-запити та побудова endpoint URL повинні бути винесені за межі page/component-рівня.

Рекомендований напрямок залежностей:

```
Page / Component
       
Repository
       
API client / HTTP
       
External API
```

Repository повинен надавати зрозумілі операції предметної області, наприклад:

```
taskRepository.getList()
taskRepository.getById(id)
taskRepository.create(data)
```

```
taskRepository.update(id, data)
taskRepository.remove(id)
```

Конкретні назви та набір методів залежать від обраної предметної області.

Компоненти та сторінки не повинні вручну будувати endpoint URL або дублювати HTTP-код.

Стилізація та BEM

Для стилізації обрати один із трьох підходів:

1. **SCSS + BEM** — власні стилі з класами за схемою Block__Element—Modifier;
2. **Bootstrap** — готові utility та component класи;
3. **Tailwind CSS** — utility-first класи.

Не потрібно поєднувати всі три підходи одночасно.

Якщо обрано **SCSS + BEM**:

- кожен компонент або логічний UI-блок має власний `.scss` файл поруч із відповідним компонентом;
- не зберігати стилі компонентів у загальному `App.css` або іншому великому спільному файлі;
- дотримуватися єдиного підходу до іменування класів;
- не будувати стилі на глибоких селекторах DOM;
- глобальні стилі залишати лише для справді глобальних речей: `body`, базові стилі, шрифти, reset тощо.

Варіанти mini-project

Вибрати одну з 10 предметних областей:

1. **Mini Shop** — каталог товарів, категорії, пошук, сторінка товару, кошик та обране.
2. **Movie Explorer** — каталог фільмів, пошук, жанри, деталі, watchlist та власні оцінки.
3. **Book Tracker** — каталог книг, пошук, деталі, статус читання та власна бібліотека.
4. **Recipe Planner** — каталог рецептів, фільтри, деталі, обране та план харчування.
5. **Travel Planner** — напрямки, пошук, деталі, збережені місця та власний маршрут подорожі.
6. **Event Planner** — каталог подій, фільтри, деталі, обране та власні події.
7. **Job Board** — список вакансій, пошук, фільтри, деталі, збережені вакансії та статус подачі.
8. **Workout Planner** — каталог тренувань, фільтри, деталі, власний план та відмітка виконання.

9. **Pet Adoption** — каталог тварин, фільтри, деталі, обране та заявки на усиновлення.
10. **Expense Tracker** — категорії витрат, список операцій, фільтри, деталі, власний бюджет та аналітика.

Для кожної теми необхідно адаптувати сценарії під її предметну область, а не механічно перейменовувати одну й ту саму реалізацію.

Предметна область може бути змінена за погодженням з викладачем.

Мінімальний функціонал

Незалежно від обраної теми застосунок повинен містити:

1. список основних сутностей;
2. пошук або фільтрацію;
3. сторінку детального перегляду;
4. щонайменше три маршрути;
5. створення або редагування однієї сутності;
6. видалення або іншу значущу CRUD-операцію;
7. окремий client/UI state;
8. асинхронний server state;
9. щонайменше одну форму з валідацією;
10. Repository для роботи з API;
11. повторно використовуваний UI-компонент, який використовується в кількох місцях.

Якщо обране зовнішнє API не підтримує потрібну mutation, дозволено реалізувати CRUD для власної client-owned сутності. Водночас робота з основним server state все одно повинна проходити через TanStack Query та Repository.

1. TanStack Query — server state

Створити `QueryClient`.

Використати щонайменше:

- `useQuery`;
- `useMutation`.

Для mutation передбачити оновлення або інвалідацію пов’язаного query.

Дані API повинні залишатися в TanStack Query. Не переносити response із API у Zustand лише для того, щоб замінити query cache.

Пошукові параметри, pagination та route params можуть бути частиною query key, якщо

вони впливають на результат запиту.

2. Zustand — client/UI state

Створити щонайменше один store.

Вибрати стан, який справді належить клієнтському застосунку.

Приклади:

- кошик;
- обране;
- watchlist;
- план подорожі;
- список збережених вакансій;
- локальні налаштування UI.

Похідні значення повинні обчислюватися зі state, а не підтримуватися окремим дубльованим state без потреби.

За бажанням можна використати persistence middleware для client-owned даних. Не потрібно зберігати в Zustand server state.

3. Forms — React Hook Form + Zod

Створити щонайменше одну форму з React Hook Form та Zod.

Форма повинна мати реальну предметну мету, наприклад:

- checkout;
- створення або редагування сутності;
- заявка;
- власна подія;
- запис витрати;
- заявка на усиновлення.

Показати валідацію помилкових даних та успішне відправлення форми.

4. Shared UI / reuse

Створити щонайменше один повторно використовуваний UI-компонент і використати його у двох або більше місцях.

Наприклад:

- `Button`;
- `Card`;
- `Modal`;
- `FormField`;
- `StatusBadge`;
- `EmptyState`.

Компонент повинен мати реальне повторне використання, а не існувати лише для виконання формальної вимоги.

5. UX-стани

Для асинхронних сценаріїв передбачити:

- loading;
- error;
- empty;
- success;
- disabled state для недоступних операцій.

Стан інтерфейсу не повинен залежати від одного універсального прапорця на кшталт `loading = true` для всіх незалежних операцій.

6. Додаткові завдання

Для підвищення оцінки реалізувати один або декілька пунктів:

1. pagination або infinite scroll;
2. optimistic update;
3. lazy routes/code splitting;
4. `React.memo`, `useMemo` або `useCallback` там, де є обґрунтована потреба;
5. адаптивну верстку;
6. темну тему;
7. unit/component tests;
8. доступність клавіатури та базові ARIA-атрибути;
9. prefetch окремих даних;
10. persistence для cart/favorites/watchlist або іншого client-owned state;
11. додаткову складну domain-specific feature.

7. Git workflow

Рекомендується feature-гілка `feature/mini-project`.

Приклад Conventional Commit:

```
feat(lab-12): build mini shop with query and zustand
```

Формат захисту

Під час захисту за короткий час необхідно продемонструвати:

1. основний користувацький сценарій;
2. роботу маршруту з параметром;
3. отримання даних через TanStack Query;
4. client/UI state через Zustand;
5. одну форму та її Zod-валідацію;
6. одну CRUD-операцію;
7. обрану структуру проєкту та пояснення відповідальності основних шарів;
8. Repository та шлях від UI до API;
9. роботу одного повторно використовуваного компонента.

Не потрібно демонструвати всі додаткові можливості.

Контрольні запитання

1. Чим server state відрізняється від client/UI state?
2. Чому TanStack Query підходить для server state?
3. Для чого потрібен QueryClient?
4. Що таке query key?
5. Коли спрацьовує refetch?
6. Що таке mutation і чим вона відрізняється від query?
7. Для чого використовується invalidation?
8. Що таке Zustand store?
9. Як компонент підписується на частину Zustand state?
10. Чому не потрібно дублювати server state у Zustand?
11. Чим feature-first відрізняється від type-first організації коду?
12. Які переваги та недоліки кожного з цих підходів?

13. Для чого потрібен Repository?
14. Чому page/component не повинен сам будувати API URL?
15. Де повинна знаходитися HTTP-логіка?
16. Яку роль виконує React Router у mini-project?
17. Як організувати форму з React Hook Form + Zod?
18. Які стани потрібно враховувати при роботі з асинхронними даними?
19. Які ознаки хорошої компонентної декомпозиції?
20. Коли виправдано застосовувати memoization?
21. Що таке code splitting і для чого він потрібен у production React?